

Complete project report · repository F:\voiceshield (private git: ryxonix/voiceshield, branch main)

Prepared: 21-Sep-2026 | Last repo audit: 19-Sep-2026 | ~4,500 lines of backend Python across 28 modules

1. What it is

VoiceShield AI is a **zero-trust, real-time audio deepfake detection and mitigation platform for the Indian telecom context**. It listens to **live voice calls** (over a WebSocket) or analyzes **uploaded audio files** and decides, in real time, whether the speaker is a **real human (bonafide)** or a **synthetic / A.I. cloned voice (spoof)**.

Target users: **telecom providers, banks, contact centers, and parents** (via a dedicated "Child Shield" mode). Everything runs on a **100% free / open-license stack** and is engineered to be **Smart India Hackathon (SIH) compliant** and **DPDP Act 2023 compliant**.

2. What problem it solves

The threat being addressed is **voice impersonation / deepfake audio fraud**: an attacker clones a victim's voice with AI and calls their family, bank, or workplace to steal money or credentials. Indian telecom networks currently have no per-call deepfake screening. VoiceShield provides:

- **Real-time verdicts** during a live call within a **sub-100 ms** per-window latency budget.
- **Explainable evidence** (prosodic XAI — jitter, shimmer, phase continuity — not just a black-box score) for human review, alerts, and law-enforcement filings.
- A **tamper-evident, legally-clean audit trail**: blockchain-anchored forensic PDFs aligned to I4C / 1930 / cybercrime.gov.in / RBI Sachet.
- **Privacy by design under DPDP**: raw audio lives only in RAM; only scalar scores leave the device or network.
- A **free stack** usable by students and startups: Telegram/Gmail/ntfy.sh/Fast2SMS/webhook alerts, free cloud GPU training, and a free-tier Oracle Cloud blockchain anchor.

3. Architecture — the detection pipeline

```
Mic / call audio → WebSocket → RingBuffer (RAM only)
| |
| (3 s window / 1 s hop; 300 ms / 100 ms fallback)
| v
| Acoustic ensemble:
| - Dhwani (Wav2Vec2 XLS-R 300m + AASIST)
| - AASIST-L official pretrained (clovaai/aasist)
| - XLSR-300m SIH cloud fine-tune (file path only)
| (all available boxes averaged)
| v
| XAI prosody layer: jitter / shimmer / phase continuity /
| pitch stability / noise-floor dropouts
| v
| Fusion: score = 0.7 x model_prob + 0.3 x xai_risk
| (enterprise watermark hit → verified, score 0.0)
| v
| Risk band → verdict → role-aware thresholds
| (adult >= 0.85 critical, child >= 0.70 critical)
v v
Scalar scores only Alerts (Telegram / Gmail / ntfy / Fast2SMS /
webhook) + incident + I4C forensic PDF
v
SHA-256 + Merkle root → PoW blockchain ledger
(optional NBF-Fabric + encrypted IPFS external anchor)
```

The pipeline is shared by both the streaming WebSocket endpoint and the file-analysis endpoint (app/engine/pipeline.py), so scoring is identical regardless of input path.

3.1 Detection engine modules (backend/app/engine)

Module	What it does
dhwani.py	Wraps the ayush2635/Dhwani ONNX detector (3 s window, 16 kHz, INT8, squid threads 2 inter / 4 intra). Logit → sigmoid spoof probability.
aasist_official.py	Loads the official clovaai/AASIST-L checkpoint (MIT). Center-pads/trims raw float32 to 64600 samples and returns spoof probability.
wfp.py	Cloud-trained XLS-R-300m Wav2Vec2ForSequenceClassification (1.2 GB safetensors). Lazy/singleton; used only on the file-analysis path.
features.py	Prosody XAI: Jitter (YIN f0, librosa.pyin), Shimmer (half-cycle peak instability), Phase Continuity (STFT phase advance), pitch stability; noise-floor dropout counter (35 dB-below-peak frames — a classic synthetic-splice artifact).
fusion.py	xai_risk = (1 - phase) x 0.5 + max(0, 1 - jitter/1) x 0.3 + max(0, 1 - shimmer/3) x 0.2; final score = 0.7 x model + 0.3 x xai. Enterprise watermark detected → short-circuit to 0.0 (verified).
ring_buffer.py	Pre-allocated NumPy circular buffer, sliding overlapping windows (reusable in place); zeroed out on reset (DPDP).
speaker.py	64-dim deterministic one-way DSP voice-print (log band energies, spectral moments, f0 stats, temporal dynamics). Cosine < 0.80 → speaker mismatch (+0.15 score). Audio cannot be reconstructed.
risk.py	Bands: critical >= role threshold; high >= threshold - 0.17; medium >= 0.35; low below. Produces verdicts and pre-transaction recommended actions.
pipeline.py	Orchestrates silence gating, ensemble model probability, watermark check, fusion, banding, and verdict into one per-window event dict.

4. Technology stack

4.1 Backend (Python 3.10+, built on 3.14.7)

Package	Purpose
FastAPI 0.141 / Uvicorn	REST + WebSocket endpoints, async ASGI server
websockets 17	Live PCM streaming during calls
NumPy / SciPy / Librosa / SoundFile	DSP: FFT, resampling, filtering, pyin f0, STFT
ONNX Runtime 1.29	INT8-quantized CPU inference (sub-100 ms/window)
PyTorch 2.14 / ONNX	AASIST-L training + ONNX export
transformers	XLS-R / Wav2Vec2 sequence-classification checkpoints
ReportLab 5	I4C-ready forensic PDF generation
httpx / aiosmtp lib	HTTP alert channels + async Gmail SMTP
cryptography	AES-256-GCM for encrypted report anchoring
pydantic-settings / python-dotenv	backend/.env configuration binding
pytest + pytest-asyncio	Unit test suite (78 tests passing)

4.2 Frontend (React 19 + Vite 8 + TypeScript 6)

Package	Purpose
React 19 + React DOM	UI framework (SPA)
Vite 8	Build tool / dev server
TailwindCSS 4	Design system / responsive styling
Custom SVG charts	RiskGauge, Radar, sparklines — zero chart libraries
Custom i18n (i18n.tsx)	English / Hindi / Kannada UI
oxlint	Linter (npm run lint)
Pages	Overview, Live call monitor, Incident log, Session forensics, Blockchain ledger, File analysis, Speaker enrollment

4.3 ML models used

Model	Role	License/source
Dhwani (Wav2Vec2 XLS-R 300m + AASIST, ONNX, ~1.2 GB)	Primary multilingual Indian deepfake detector (hi/en/kn/ta/te/ml)	MIT weights (ayush2635, HF)
AASIST-L official (clovaai/aasist)	Production ensemble member / countermeasure backbone (<2 MB)	MIT
AASIST-L local (ours, fine-tune, ONNX ~0.1 MB)	Offline fallback detector; neutral 0.5 until re-exported	MIT (ours)
XLSR-300m SIH cloud fine-tune (1.2 GB safetensors)	Cross-lingual spoof classifier from cloud notebook 03	ours (base Apache-2.0)
Speaker voice-print (64-dim one-way embedding)	Cross-session speaker consistency (DPDP-safe)	MIT (ours)

4.4 Cloud training & deployment

Platform/tool	Purpose
Google Colab (free T4)	colab_01_build_dataset / colab_02_train_wav2vec2 / colab_03_train_xlsr
Kaggle (T4 x2)	kaggle_train_wav2vec2.py equivalent
Hugging Face Hub (optional)	Dataset / model push + reload
deploy/edge/vs_edge.py	On-device CLI inference worker — no GPU/cloud dependency
deploy/nbf-fabric/	Hyperledger Fabric 2.2 + Kubo IPFS + Node gateway on Oracle Cloud Always-Free ARM or WSL2

5. Key features

- **Multi-layer detection, not one model** — ensemble acoustic models AND explainable prosody combined: $0.7 \times \text{model} + 0.3 \times \text{XAI}$.
- **Real-time by design** — WebSocket PCM streaming; 3 s windows / 1 s hop (300 ms / 100 ms fallback); only scalar scores cross the wire (DPDP-safe).
- **Free alerts everywhere** — Telegram, Gmail SMTP, ntfy.sh, Fast2SMS (India), generic webhook — all optional, missing keys skipped at runtime.
- **Role-aware risk thresholds** — adult ≥ 0.85 critical, child ≥ 0.70 critical (configurable in .env).
- **Child Shield** — lower threshold, auto-mute, guardian/authority notification, protective overlay.
- **Contextual enrichment** (opt-in, CONTEXTUAL_ENRICHMENT=true, fail-open) — caller / known-contact / origin / txn-value / historical flags adjust risk before thresholding.
- **Configurable mitigation workflows** (workflows.json, JSON — no code deploy) — role x risk-band → actions + channels; POST /api/incidents/{iid}/escalate feeds caller reputation.
- **DoT hand-off** — suspected-fraud metadata to Sanchar Saathi / Chakshu / DIP via CHAKSHU_DIP_WEBHOOK_URL; confirmed victims directed to 1930 / cybercrime.gov.in / RBI Sachet in the PDF.
- **I4C-ready forensic PDFs** — IST timestamps and legal next steps.
- **Tamper-evident integrity** — every PDF: SHA-256 + Merkle root over per-window evidence mined into a local proof-of-work SHA-256 ledger (VoiceShieldAIV1, difficulty 4).
- **NBF anchor (MeitY Vishvasya)** — each block also committed to Hyperledger Fabric via an NBF-Lite-compatible REST gateway; the report PDF is AES-256-GCM encrypted and pinned on IPFS as ciphertext (only hashes/CID/key-fingerprint on-ledger). Fail-open: gateway down → pending/retryable.
- **Speaker ID** — enrollment + cross-session voice-print consistency (one-way, unreconstructable).
- **Privacy** — ephemeral RAM processing, zero disk for raw audio, scalar-only telemetry; DPDP helpers in app/compliance/dpdp.py.
- **i18n** — en / hi / kn dashboard.
- **Languages** — tuned for Hindi, English, Kannada (+ Tamil/Telugu/Malayalam via Dhvani).
- **SDK + proto contract** — voiceshield_sdk (sync + async) on voiceshield.proto (proto3); REST + live WebSocket call sessions.
- **Edge worker** — deploy/edge/vs_edge.py straightforwardly proves the on-device inference option.

6. REST API surface

Endpoint	Function
POST /api/analyze	Full sliding-window analysis of an uploaded file (accepts optional context fields: caller, origin, txn_value, txn_category, known_contact, prior_flags)
POST /api/detect	Quick single-shot detection on an audio payload
WS /ws/stream/{call_id}	Live streaming detection during a call (PCM in, JSON events out)
GET /api/sessions/{sid}/windows[]	List sessions / detail / per-window telemetry
GET /api/incidents/{iid}[] /ack[] /escalate[] /report[]	List incidents; acknowledge; escalate (bump caller reputation + webhook); I4C forensic PDF
GET /api/report/{call_id}	Forensic PDF by call id
GET/POST /api/blockchain[...]	Ledger status, blocks, verify/block, verify/call, onchain summary, onchain/{call_id}, retry/{call_id}

Endpoint	Function
POST /api/speakers/register	Enroll a speaker voice-print (one-way embedding)
GET /api/workflows	Current mitigation workflow rules (role x band)
GET /health , /info	Health and model/configuration status

7. Training data & licensing (SIH-compliant)

7.1 Bonafide (real human voices)

Source	License	Why used
Google FLEURS (hi_in / en_us / kn_in + ta/te/ml/mr)	CC-BY-4.0	Real single-speaker read-speech at 16 kHz — the anchor "genuine" class
Mozilla Common Voice 17 (top-up)	CC0 (<=v13) / CC-BY-4.0 (v14+)	Enlarges the real-voice pool in the cloud pipeline

7.2 Spoof / deepfake generation (real A.I. impersonation)

Tool	License	How used
Coqui XTTS-v2	CPML-1.0 (non-commercial)	Clones the actual bonafide speakers speaking attacker-style phrases — the real SIH voice-impersonation attack
RVC / FreeVC / FreeVC24	MIT	Voice-conversion style clones (second major deepfake family)
edge-tts / gTTS	free (unofficial endpoints)	Plain TTS minority attack class (robocalls); eval/demo only, never shipped
Offline voiced-noise fallback	ours	Guarantees a spoof class even offline
ASVspoof 2019 LA (reference only)	research	External benchmark — NOT bundled/redistributed in the repo

7.3 Telecom-channel augmentations

G.711, AMR-NB/WB (optional ffmpeg), packet-loss, bandwidth, and noise — simulating degraded Indian mobile call channels.

7.4 Training paths

On-device: `training/fetch_indic` → `build_sih_dataset` → `merge_protocols` → `train` → `export_onnx` (local checkpoint log: 2 epochs, val EER 0.1763, saved to `checkpoints/best_model.pth`).

Cloud (recommended, free GPU): Google Colab + Kaggle notebooks in cloud/— dataset build with real clones, wav2vec2-base and XLS-R-300m fine-tunes. Artifacts saved under `voiceshield_cloud/` (train.csv / val.csv / model.safetensors).

7.5 Integrated services (all free / free-tier)

Service	Function	Config key(s)
Telegram Bot	Real-time alert messages	+
Gmail SMTP (App Password)	Email alerts	SMTP_USER / SMTP_PASSWORD / ALERT_EMAIL_TO
ntfy.sh	Push notifications, no signup	NTFY_TOPIC

Service	Function	Config key(s)
Fast2SMS	SMS to Indian mobiles	FAST2SMS_API_KEY / FAST2SMS_TO_NUMBER
Webhook	Any endpoint you control	WEBHOOK_URL
Resend / Sentry	Alt email / error tracking (optional)	RESEND_API_KEY / SENTRY_DSN
Neon PostgreSQL (optional)	Serverless multi-machine persistence (else SQLite/in-memory)	DATABASE_URL
Hugging Face Hub (optional)	Model/data download + push	HF_TOKEN
Local PoW SHA-256 ledger	Tamper-evident report anchoring (no key needed)	BLOCKCHAIN_DIFFICULTY
NBF-Lite anchor (opt-in)	Hyperledger Fabric + IPFS encrypted anchor	BLOCKCHAIN_EXTERNAL_ANCHOR / NBF_GATEWAY_URL
DoT escalation (optional)	Sanchar Saathi / Chakshu / DIP suspected-fraud feeding	CHAKSHU_DIP_WEBHOOK_URL

8. Project history (git log, newest first)

Commit	Summary
2f9536e	feat(SIH26104): contextual enrichment, configurable workflows, SDK+proto contract, edge worker
b42f9cf	docs: NBF live end-to-end verification evidence + README pointer
a32e4b3	chore: untrack regenerable aasist_l.onnx.data blob
61e144e / bc6534e / d000d60 / 5b49385 / 4a99441	Codespaces + deploy fixes and repo audit / truth pass
7082d72 ... c944211 (~15 commits)	Incremental Hyperledger-Fabric deployment fixes: cryptogen identities, configtx Orderer policies, chaincode install idempotency, peer 2.2 Alpine quirks, gateway IPFS native-RPC, devcontainer fixes
a013744	NBF-Lite on-chain anchor, engine/frontend fixes, gender feature removed
fe08163	Initial commit: VoiceShield AI full project structure

8.1 Current work-in-progress (git status)

- New gRPC package: backend/app/grpc/ (server.py, servicer.py, voiceshield_pb2[_grpc].py).
- SDK work: sdk/_pb2_backup/, sdk/_regen/, voiceshield_sdk/grpc.py, gen_probe(2).py scripts.
- New tests: test_grpc_aio_aclose, test_latency_gate, test_parity_live.
- Modified: backend/app/config.py. This is the next milestone — exposing the shipped voiceshield.proto contract as a real gRPC server (+ latency gates and parity tests).

9. Current status & verified results

- 78 pytest tests passing (backend/tests/: blockchain, context enrichment, external anchor, features, fusion, latency gate, parity live, PDF report, ring buffer, SDK, websocket, workflows, gRPC-aio).
- NBF external anchor LIVE-verified on 2026-09-19: call e2e-NBF-LIVE-01 — GET /api/blockchain/onchain/e2e-NBF-LIVE-01 → anchored, verified:true, problems:[]. Fabric tx-id tx-test-1; IPFS CID QmZimdoSUNhXCrt...48kB; local chain VoiceShieldAIV1 height 14, difficulty 4, valid:true. Evidence in docs/nbf-live-demo.md.
- Generated forensic PDF artifacts under backend/reports/ (VoiceShield_Forensic_Report_*, VoiceShield_Incident_*); test_report.pdf.
- Runtime: Python 3.14.7 venv at backend/venv; Node v26.9.0 / npm 11.19.1; optional ffmpeg only for AMR augmentation; SQLite at backend/voiceshield.db (or in-memory / optional Neon).
- Latency budget (per 3 s window): ingest 7 ms + prosody 20 ms + INT8 inference 35 ms + watermark/fusion/mitigation 16 ms = 78 ms total.
- Note: on a fresh clone, git-ignored artifacts (checkpoints/*.pth, models/aasist_l.onnx, models/dhwani/) must be present or re-exported; until then fallback inference is neutral 0.5. Dhwani + official AASIST-L load from models/ + hf_models/ when present.

10. How to run

```
# 1) Backend
cd backend
python -m venv venv && venv\Scripts\activate
pip install -r requirements.txt

# 2) Configure keys
copy .env.example .env # add free keys: Telegram, Gmail app password, ntfy,
# Fast2SMS, webhook (+ optional Resend/Sentry/Neon)
```

```
# 3) Start backend
python -m uvicorn app.main:app --host 0.0.0.0 --port 8000
# API docs -> http://localhost:8000/docs

# 4) Start frontend
cd frontend && npm install && npm run dev
# Dashboard -> http://localhost:5173

# 5) One-click (Windows)
start_all.bat # starts backend + frontend, skipping busy ports
```

Optional: re-export the ONNX fallback with `python -m app.models.export_onnx checkpoints/best_model.pth`; train on-device with `python -m training.train ...`; train in the cloud with the Colab/Kaggle notebooks in cloud/.

Source: README.md, about project.txt, backend/SOURCES_AND_TECHNOLOGY.md, backend/SDK_README.md, working-tree code under F:\voiceshield. Project code is MIT; third-party licenses retained (FLEURS CC-BY-4.0, Common Voice CC0/CC-BY-4.0, AASIST MIT, Dhvani MIT, etc.) — full inventory in SOURCES_AND_TECHNOLOGY.md.